



Métodos de Pesquisa e Ordenação em Estruturas de Dados

UNIDADE 04

Estrutura de dados: Grafos

Olá!

Seja bem-vindo a um dos tópicos mais intrigantes e essenciais no universo da ciência da computação, e que tem uma fundamentação bem forte na matemática: os grafos. Pode ser que você esteja se perguntando:

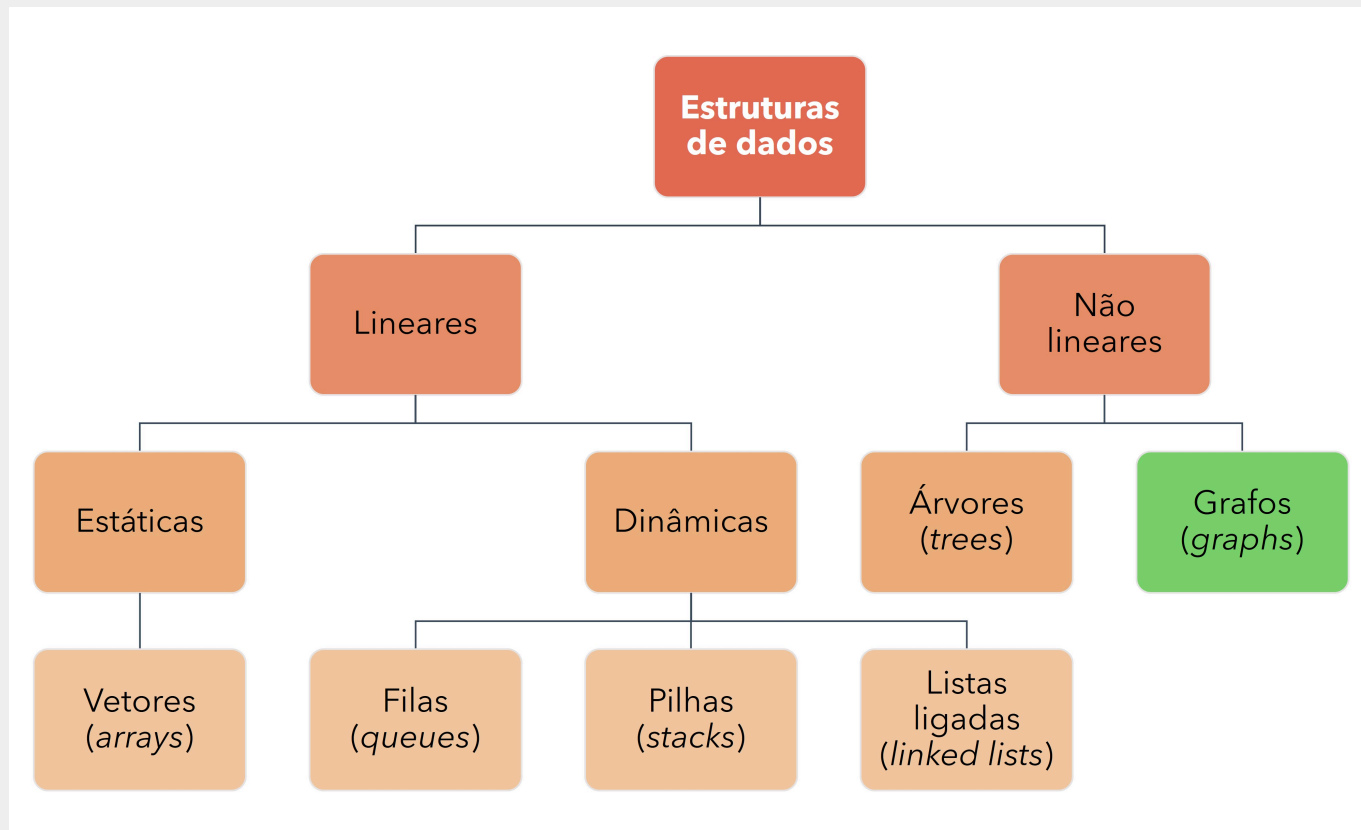
#ParaTodosVerem 



A resposta é simples: vivemos em um mundo cheio de conexões entre coisas, pessoas e sistemas, e os grafos são a representação perfeita dessa interconexão. Em vários casos, eles são a solução para evitar que você faça várias gambiarras em seu código para tentar representar conceitos mais complexos. E, sabe que tipo de profissional as empresas pagam bem? Pois é: aqueles que **não fazem muita gambiarra**.

Pense nos diversos sistemas e aplicações que manipulamos diariamente. Seja nas redes sociais, mapeando relações entre indivíduos; em sistemas de recomendação, conectando usuários a produtos; ou em sistemas de roteamento, ligando pontos em um mapa. Todos esses exemplos têm algo em comum: a necessidade de representar e trabalhar com entidades interligadas. E é aqui que os grafos brilham, proporcionando uma estrutura que nos permite não só representar, mas também **analisar** e **otimizar** essas conexões.

Para analistas de sistemas e desenvolvedores de *software*, dominar o conceito de grafos e suas aplicações não é apenas uma habilidade acadêmica, mas, sim, uma ferramenta muito útil no *design* e implementação de soluções de *software* que sejam robustas e eficientes. Em uma era pós-pandemia a capacidade de analisar e otimizar relações complexas é um diferencial competitivo. Portanto, ao se aprofundar no estudo de grafos, você estará não apenas ampliando seus horizontes teóricos, mas também adquirindo uma competência essencial para resolver desafios práticos no mundo real da análise de sistemas. Bora lá?



Fonte: O autor (2023).

Ao abordarmos o universo dos grafos, descobrimos um território cheio de aplicações e soluções dentro das empresas. Estas estruturas, que consistem em nós (ou vértices) interconectados por arestas, são soluções para **problemas que envolvem conexões e relações**. Agora, sendo mais direto: em que casos usamos os grafos nas empresas?

Imagine, por exemplo, um sistema empresarial de gestão de projetos. Para otimizar a alocação de recursos e mapear as dependências entre diferentes tarefas, podemos recorrer a grafos. Ao representar cada tarefa como um nó e suas dependências como arestas, é possível identificar o melhor caminho para a conclusão do projeto, evitando gargalos e maximizando a eficiência.

Engenheiros e cientistas de dados frequentemente enfrentam o desafio de criar sistemas de recomendação, como aqueles que sugerem produtos em lojas online com base no histórico de compras ou na navegação do usuário. Grafos, neste contexto, são essenciais para mapear as relações entre produtos e consumidores, permitindo a geração de recomendações precisas e personalizadas. Em outro cenário, quando pensamos em redes corporativas, engenheiros de *software* e analistas de sistemas, frequentemente utilizam grafos para modelar e analisar fluxos de informação, identificar pontos de falha ou otimizar caminhos de comunicação.

Para desenvolvedores de *software*, os grafos também são indispensáveis em tarefas como análise de dependência em sistemas de *software* complexos. Eles podem ajudar a visualizar e entender como diferentes módulos ou serviços interagem entre si, garantindo integridade e eficiência ao sistema como um todo. Em outras palavras: ajudam a entender qual sistema está devagar ou causando um gargalo para os demais.

Neste momento você ainda pode estar se perguntando:



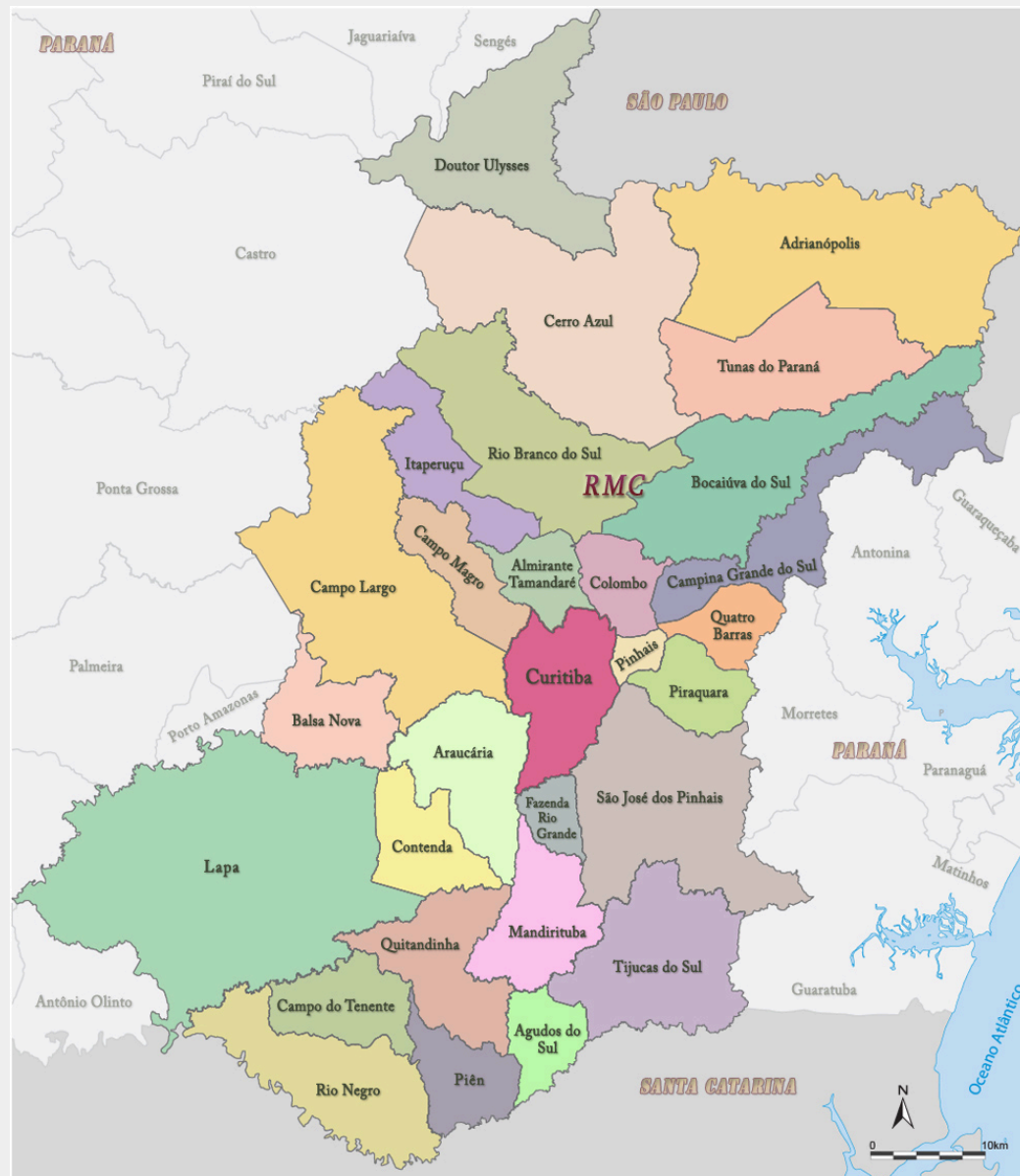
Vamos supor que você recebeu uma ligação de uma dona de uma empresa de logística da sua região. Ela lhe pagará o valor de uma casa nova se você desenvolver um sistema para planejar as rotas da empresa dela. Legal, né? Agora, pense um pouco: como você desenvolveria um sistema para encontrar a melhor rota com o seu conhecimento de listas, vetores, pilhas e filas? Como você armazenaria em seu sistema todas as cidades que se conectam umas às outras? Acabei de descrever para você o problema do **caixeiro-viajante**, bem comum nesta área. E aí? Vai perder o dinheiro de uma casa novinha só porque não aprendeu grafos?



| Estrutura de Dados: Grafo

Agora, o que é um grafo? Pense nas matrizes: os grafos podem ser representados usando uma estrutura chamada "matriz de adjacência". Imagine uma matriz em que as linhas e as colunas representam os vértices (pontos) do grafo. Se dois vértices são conectados por uma aresta, então a célula correspondente na matriz terá o valor 1; caso contrário, terá o valor 0. Assim, essa matriz nos dá uma representação visual de

todas as conexões no grafo. Para explicar melhor, veja um mapa da Região Metropolitana de Curitiba:



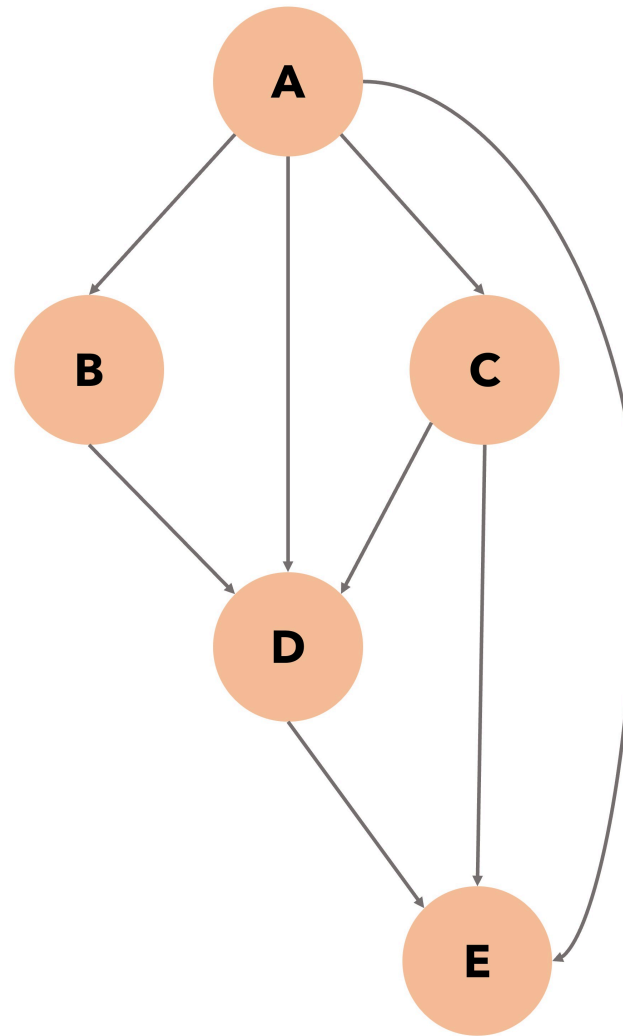
Mapa da Região Metropolitana de Curitiba. Fonte: AMEP - Agência de Assuntos Metropolitanos do Paraná.

Veja que algumas cidades estão conectadas às outras: Curitiba está ao lado de cidades como São José dos Pinhais, Colombo e Araucária, mas não faz nenhuma divisa com Rio Branco do Sul ou Quatro Barras, por exemplo. Vamos representar as conexões entre algumas dessas cidades em uma matriz, em que 1 representa que há uma conexão (divisa) e 0 representa que não há uma divisão:

	Curitiba	Araucária	Pinhais	Colombo	Quatro Barras	Itaperuçu	Campo Largo
Curitiba	0	1	1	1	0	0	1
Araucária	1	0	0	0	0	0	1
Pinhais	1	0	0	1	1	0	0
Colombo	1	0	1	0	1	0	0
Quatro Barras	0	0	1	1	0	0	0
Itaperuçu	0	0	0	0	0	0	1
Campo Largo	1	1	0	0	0	1	0

Este tipo de matriz se chama **matriz de adjacências** e é uma das formas mais simples de explicar grafos. Por outro lado, ela não é exatamente escalável: em TI, é comum termos grafos com milhares (ou dezenas de milhares) de elementos. Portanto, usaremos este tipo de representação apenas para fins de aprendizagem.

Agora, **o que é um grafo?** Vejamos a imagem a seguir como um exemplo: ela é uma representação visual de um grafo.



Exemplo de grafo contendo cinco vértices (nós) e oito arestas (conexões).
Fonte: O autor (2023).

São os círculos A, B, C, D e E. São elementos que podem se conectar uns aos outros. Estes nós podem ser cidades, sistemas de recomendação, tabelas de banco de dados ou pessoas em redes sociais, por exemplo.

+ Arestas (ou conexões)

Estabelecem quais nós conectam-se uns aos outros. Veja que nem todos nós se conectam com todos os outros nós. Por exemplo: o nó C se conecta com A, D e E, mas não se conecta com B.

+ Direção

Veja que as conexões possuem flechas. Isto indica que se trata de um grafo **direcionado**. É possível irmos do nó A ao nó C (veja a flecha saindo de A para C), mas não é possível fazermos o movimento inverso. Se não houvesse estas flechas teríamos um grafo **não direcionado**.

+ Peso

Também é possível colocarmos um valor para cada aresta. Pensemos no caso de cidades: pode ser interessante informar não somente quais cidades estão conectadas umas às outras, mas também a distância entre o centro da

cidade de cada uma delas. No caso, esta distância seria um exemplo deste peso. Assim, saberíamos que alguns caminhos são mais curtos do que outros.

Implementando grafos no Java

O pacote-padrão do Java não possui uma implementação padrão para grafos (ao menos não tinha enquanto estávamos escrevendo isso). Logo, para fins de teste, implementaremos a nossa própria lógica usando duas classes do Java: ***ArrayList*** (vetores) e ***LinkedList*** (listas encadeadas). Essas duas classes serão usadas para simularmos uma matriz de adjacência em nossos exemplos. E, para testar, vamos usar como ponto de partida aquele exemplo do grafo contendo as ligações entre algumas cidades da Região Metropolitana de Curitiba.



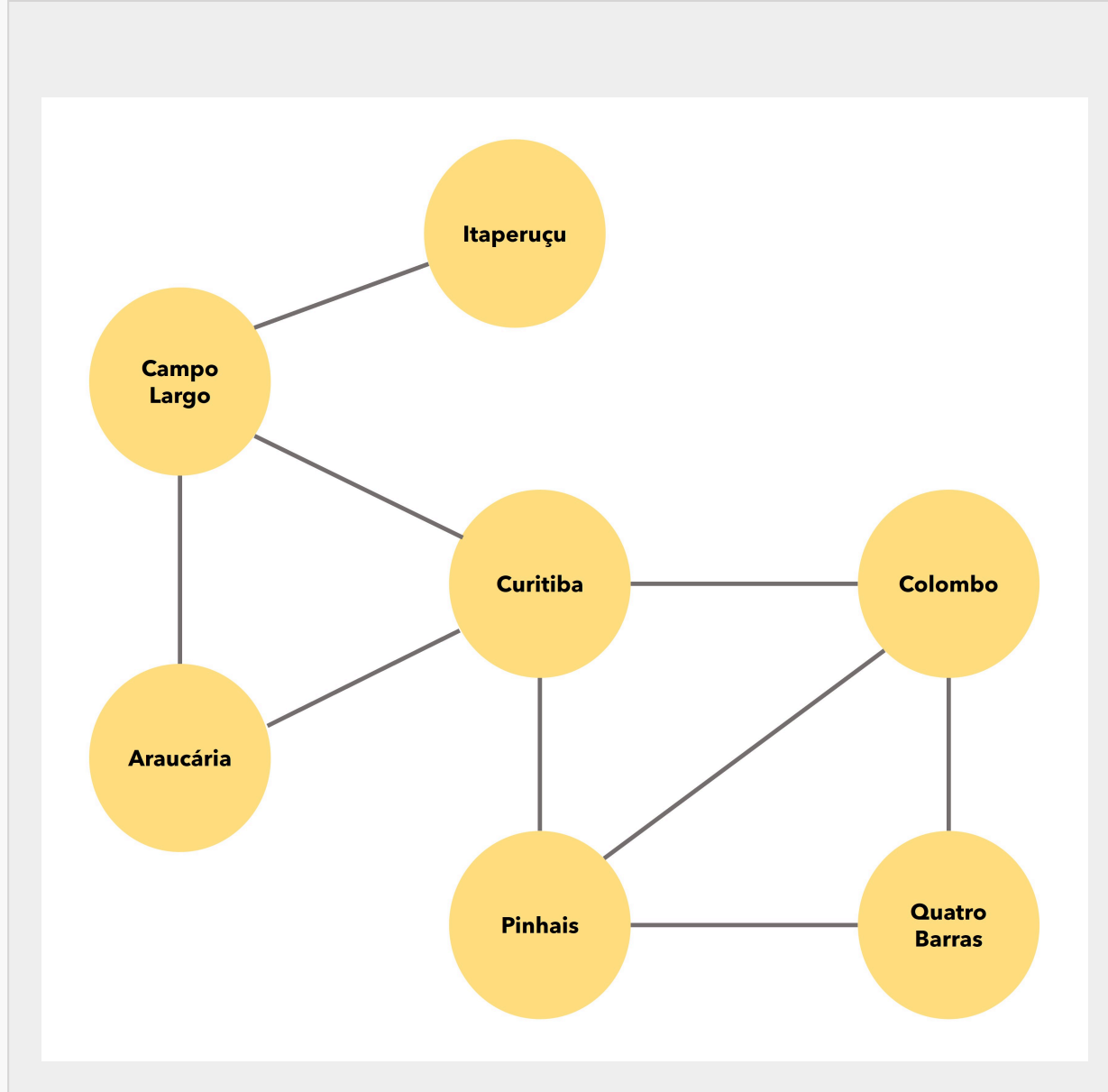
Testaremos os seguintes tipos:

1. Grafos sem direção e sem peso.
2. Grafos sem direção e com peso.
3. Grafos direcionados e sem peso.
4. Grafos direcionados e com peso.

GRAFOS SEM DIREÇÃO E SEM PESO

O nosso grafo será parecido com a figura a seguir.

#ParaTodosVerem 



Grafo contendo algumas cidades da Região Metropolitana de Curitiba. Fonte: O autor (2023).

Agora, como implementaríamos isto? Teste o código a seguir, escrito em Java:

```
</>
```

```
1 import java.util.ArrayList;
2 import java.util.LinkedList;
3
4 class Grafo {
5     private ArrayList<String> cidades;
6     private ArrayList<LinkedList<String>> rotas;
7
8     public Grafo() {
9         cidades = new ArrayList<>();
10        rotas = new ArrayList<>();
11    }
12
13    public void adicionarCidade(String cidade) {
14        cidades.add(cidade);
15        rotas.add(new LinkedList<>());
16    }
17
18    public void adicionarRota(String origem, String destino) {
19        int indexOrigem = cidades.indexOf(origem);
```

Agora, qual seria o resultado deste código? Recomendo **muito** que o teste, mas o resultado será isto:

```
A cidade de Curitiba tem rotas para: Araucária, Pinhais, Colombo, Campo Largo
A cidade de Araucária tem rotas para: Curitiba, Campo Largo
A cidade de Pinhais tem rotas para: Curitiba, Colombo, Quatro Barras
A cidade de Colombo tem rotas para: Curitiba, Pinhais, Quatro Barras
A cidade de Quatro Barras tem rotas para: Pinhais, Colombo
A cidade de Itaperuçu tem rotas para: Campo Largo
A cidade de Campo Largo tem rotas para: Curitiba, Araucária, Itaperuçu
```

E como nós acabamos de implementar o grafo? Pense neste grafo como um mapa das cidades de uma região e suas estradas interconectadas. Se quisermos saber se podemos viajar de uma cidade a outra por estrada, podemos consultar este grafo. Adicionando cidades e rotas, estamos basicamente construindo este mapa passo a passo. E ao imprimir o grafo, temos um resumo das conexões entre todas as cidades no mapa.

Imagine este grafo como sendo um mapa. Cada cidade no mapa é um **vértice** (ou nó), e as estradas que conectam essas cidades são as **arestas** (ou conexões). Veja que:

1. **Linhas 4-42:** começamos a implementar uma classe chamada Grafo para comportar todas as nossas operações.
2. **Linha 5:** o grafo precisa guardar os nomes das cidades (ou vértices, ou nós). Neste caso, pode ser uma lista de nomes das cidades.
3. **Linha 6:** o grafo precisa também guardar as conexões entre cidades. Aqui, **rotas** é uma lista de listas. Cada elemento desta lista corresponde a uma cidade em **cidades**, e dentro de cada lista temos todas as cidades conectadas a ela por uma estrada (aresta).
 1. Exemplo: se a primeira cidade se chama “Curitiba”, o primeiro elemento dentro de *rotas* possui todas as cidades que se conectam a Curitiba.
4. **Linhas 8-10:** quando criamos um grafo, as listas cidades e rotas são inicializadas vazias. Isto é um construtor (que *aprendemos* lá em Fundamentos de Programação Orientada a Objetos).
5. **Linhas 13-16** (método **adicionarCidade**): ao adicionar uma cidade, adicionamos o nome da cidade à lista cidades e uma nova lista vazia à lista rotas para guardar futuras conexões dessa cidade.
6. **Linhas 18-26** (método **adicionarRota**): este método estabelece uma conexão (rota) entre duas cidades. Primeiro, encontramos os índices das cidades origem e destino na lista cidades. Em seguida, se ambas as cidades existirem (índices não são iguais a -1), adicionamos a cidade destino à lista de rotas da cidade origem e vice-versa.
 1. Adicionamos duas vezes (a cidade destino às rotas da origem, e a cidade de origem às rotas do destino) porque o grafo é não direcionado. Ou seja: a rota é bidirecional – podemos tanto ir quanto voltar.
7. **Linhas 28-41** (método **imprimirGrafo**): este método exibe cada cidade e suas respectivas conexões (rotas). Ele percorre cada cidade e imprime o nome da cidade seguido por todas as cidades conectadas a ela.
8. **Linhas 44-66** (classe Main e método **main**): aqui, estamos demonstrando o uso da classe Grafo. Primeiramente, criamos um grafo. Em seguida, adicionamos várias cidades e estabelecemos rotas entre elas. No fim, mostramos o grafo para visualizar as conexões.

Você pode ter percebido que esta lógica é mais complexa em relação às outras estruturas que vimos até o momento, não é? Logo, vamos fazer uma pequena pausa para uma revisão.

Entendendo quando usar grafos

Os grafos são muito poderosos para modelar e resolver problemas que envolvem **relações complexas, ou não lineares**. Já as listas, vetores, pilhas e filas têm suas próprias forças em cenários mais **lineares e ordenados**. Vejamos algumas diferenças entre as estruturas que vimos até agora:

+ Quanto ao funcionamento geral destes tipos de dados:

- **Grafos:** uma estrutura de dados **não linear** que representa relações entre pares de elementos. Seus elementos são chamados de vértices (ou nós) e as relações são representadas por arestas.
- **Listas e vetores:** são estruturas **lineares** onde a inserção e remoção são feitas em posições específicas.
- **Pilhas:** uma estrutura **linear** que segue o princípio de *Last-in, First-out* (LIFO). O último elemento inserido é o primeiro a ser removido.
- **Filas:** uma estrutura **linear** que segue o princípio de *First-in, First-out* (FIFO). O primeiro elemento inserido é o primeiro a ser removido.

+ Quando é bom usarmos grafos?

- Se quisermos representar **relações (ou relacionamentos)** entre os elementos. Os grafos são ideais para representar relações binárias entre pares de objetos, como redes de computadores, sistemas de recomendação, redes sociais e mapas de rotas.

- Se quisermos **flexibilidade** na representação: os grafos podem ser direcionados, não direcionados, ponderados ou não ponderados, dando a eles uma versatilidade que listas e vetores não possuem.

+ Quando não é legal usarmos grafos?

- Quando temos **pouca memória** ou menos **experiência**: são estruturas mais complexas que precisam de mais memória e, como percebemos, podem ser mais difíceis de serem implementadas corretamente pela sua complexidade.
- Operações de **busca**: buscar um vértice ou aresta específica pode ser mais demorado em um grafo do que em um vetor ou lista, especialmente se o grafo não for bem-estruturado.

+ Outras comparações:

- Geralmente usamos listas, vetores, pilhas e filas são estruturas para armazenar e acessar dados de maneira **ordenada**. Já em grafos, isso acaba não sendo uma prioridade.
- Lembra que tínhamos **regras** bem específicas para inserir e remover elementos em pilhas e filas? Em grafos não temos aquele tipo de regra.

- Procurar informações em listas ligadas pode ser **lento**. Já em grafos, pode ser algo bem **rápido**.

GRAFOS SEM DIREÇÃO E COM PESO

Bom, o que acha de modificarmos aquele grafo para inserirmos um peso nas arestas? Por exemplo: e se quiséssemos representar a distância em quilômetros entre o centro destas cidades em cada aresta? Vejamos como ficou o código modificado em relação ao exemplo anterior:

</>

```
1  import java.util.ArrayList;
2  import java.util.LinkedList;
3
4  class Rota {
5      String destino;
6      int peso;
7
8      public Rota(String destino, int peso) {
9          this.destino = destino;
10         this.peso = peso;
11     }
12 }
13
14 class Grafo {
15     private ArrayList<String> cidades;
16     private ArrayList<LinkedList<Rota>> rotas;
17
18     public Grafo() {
19         cidades = new ArrayList<>();
```

Beleza, mas o que mudou em relação a este exemplo e o anterior? Veja só:

1. **Linhas 4-12:** tivemos de implementar uma nova classe chamada **Rota** para representar a conexão entre duas cidades e, ainda, o peso associado a essa conexão.
2. **Linha 16:** a nossa lista de rotas passou a usar a nossa nova classe **Rota** em vez de *strings* representando as cidades.
3. **Linhas 28, 33 e 34:** adaptamos o método **adicionarRota** para lidar com a nova classe **Rota**. Veja que agora estamos incluindo o valor do peso (no nosso caso, a distância em km) a cada rota.
4. **Linha 45:** adaptamos o método **imprimirGrafo** para que também mostre a distância em km entre as cidades.
5. **Linhas 67-75:** toda vez que adicionamos uma nova rota também estamos informando como último parâmetro a distância em km entre as duas cidades.

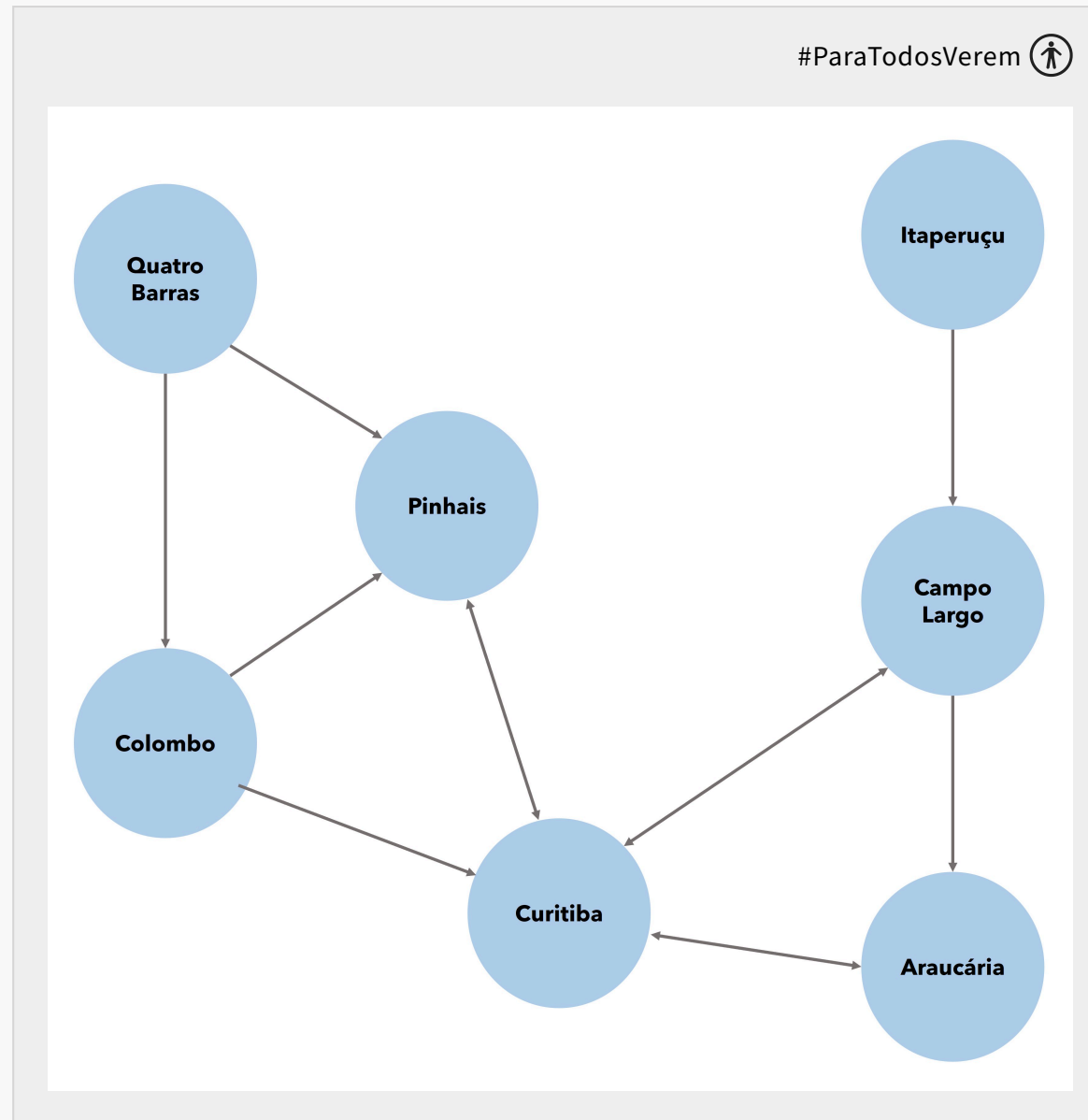
O resultado do código será como aparece no exemplo a seguir. Veja que agora temos as distâncias para cada caso.

```
A cidade de Curitiba tem rotas para: Araucária (distância: 28 km), Pinhais (distância: 9 km), Colombo (distância: 13 km), Campo Largo (distância: 30 km)
A cidade de Araucária tem rotas para: Curitiba (distância: 28 km), Campo Largo (distância: 28 km)
A cidade de Pinhais tem rotas para: Curitiba (distância: 9 km), Colombo (distância: 9 km), Quatro Barras (distância: 18 km)
A cidade de Colombo tem rotas para: Curitiba (distância: 13 km), Pinhais (distância: 9 km), Quatro Barras (distância: 14 km)
A cidade de Quatro Barras tem rotas para: Pinhais (distância: 18 km), Colombo (distância: 14 km)
A cidade de Itaperuçu tem rotas para: Campo Largo (distância: 42 km)
A cidade de Campo Largo tem rotas para: Curitiba (distância: 30 km), Araucária (distância: 28 km), Itaperuçu (distância: 42 km)
```

GRAFOS DIRECIONADOS E SEM PESO

E se quiséssemos trabalhar com grafos direcionados? Quer dizer: grafos nos quais não necessariamente você pode *ir e voltar*, mas somente ir a uma direção dependendo do caso. Para ilustrar isso, vamos considerar um exemplo *totalmente fictício* em que estamos tentando representar a principal tendência de fluxo de trânsito durante as manhãs entre as cidades. Por exemplo: Araucária tem muitas indústrias, e é comum termos

trabalhadores de Curitiba e Campo Largo indo para lá trabalhar. Por outro lado, Curitiba também tem muitas empresas, e é igualmente comum termos trabalhadores de Araucária fazendo o movimento inverso. Já Itaperuçu é uma cidade bem pequena: é comum termos pessoas desta cidade indo para Campo Largo, que é maior. Já o contrário não ocorre com o mesmo volume. Para melhor representar isto vejamos a imagem a seguir:



Grafo direcionado contendo algumas cidades da Região Metropolitana de Curitiba. Fonte: O autor (2023).

Agora, como adaptar o nosso código para comportar isto? Para transformar o grafo de não direcional para direcional, precisamos fazer apenas uma alteração principal: quando adicionamos uma rota de uma cidade "origem" para uma cidade "destino", não devemos mais adicionar a rota inversa automaticamente.

No grafo não direcional, quando você adiciona uma rota de "Curitiba" para "Araucária", a rota de "Araucária" para "Curitiba" também é adicionada. No grafo direcional, essa rota inversa não é adicionada, tornando a relação unidirecional. Vamos usar o código que fizemos dentro da seção **Grafos sem direção e sem peso** como base. Veja o código modificado, a seguir. Fizemos as seguintes mudanças:

- 1. **Linha 23:** no método **adicionarRota**, estamos somente adicionando a cidade de origem ao destino. Veja que no código original fazíamos este processo de adição de uma cidade a uma rota duas vezes: agora, é somente uma vez.
- 2. **Linhas 55-66:** precisamos atualizar as rotas para que elas contivessem somente as direções corretas. Por isso, veja que atualizamos as origens e destinos para refletir exatamente o grafo que vimos anteriormente.

Execute o código a seguir. Veja as mudanças na listagem das rotas: veja que nem todas as origens e destinos compartilham a mesma rota. Por exemplo: você pode ir de Colombo para Pinhais, mas não o contrário.

</>

```
1 import java.util.ArrayList;
2 import java.util.LinkedList;
3
4 class Grafo {
5     private ArrayList<String> cidades;
6     private ArrayList<LinkedList<String>> rotas;
7
8     public Grafo() {
9         cidades = new ArrayList<>();
10        rotas = new ArrayList<>();
11    }
12
13    public void adicionarCidade(String cidade) {
14        cidades.add(cidade);
15        rotas.add(new LinkedList<>());
16    }
17
18    public void adicionarRota(String origem, String destino) {
19        int indexOrigem = cidades.indexOf(origem);
```

GRAFOS DIRECIONADOS E COM PESO

E se fôssemos construir grafos com direção e com peso? Bom: basicamente, combinaremos o aprendizado dos grafos sem direção e com peso (quando incluímos uma classe a mais, chamada **Rota**) e a atualização que fizemos em **adicionarRota** dentro da seção **Grafos direcionados e sem peso**. Logo, veja como ficaria o código dessa maneira:

</>

```
1 import java.util.ArrayList;
2 import java.util.LinkedList;
3
4 class Rota {
5     String destino;
6     int peso;
7
8     public Rota(String destino, int peso) {
9         this.destino = destino;
10        this.peso = peso;
11    }
12 }
13
14 class Grafo {
15     private ArrayList<String> cidades;
16     private ArrayList<LinkedList<Rota>> rotas;
17
18     public Grafo() {
19         cidades = new ArrayList<>();
```

Veja que as alterações que fizemos nas duas últimas sessões foram unificadas aqui. Perceba também que:

1. **As linhas 66 a 69:** coloquei pesos diferentes quando mudamos a origem e destino. Isto é: a distância de Curitiba para Araucária não é a mesma distância de Araucária para Curitiba. Assim como na realidade, não necessariamente queremos que todos os caminhos tenham o peso em todas as direções. Em grafos, é possível colocarmos este tipo de nuance no código.

| Conclusão

Chegamos ao fim da nossa unidade sobre grafos, e espero que você tenha compreendido o valor e a profundidade deste tópico. A teoria dos grafos, apesar de sua simplicidade aparente, é uma ferramenta poderosa que se estende a muitas áreas dentro de TI e fora dela também. Veja só: como comentei anteriormente, dentro das empresas os grafos desempenham um papel bem relevante em uma variedade de aplicações. Desde otimização de rotas em sistemas de logística até a análise de redes sociais para *marketing*, passando por sistemas de recomendação em plataformas online e muitas outras aplicações.

Por isso, compreender o conceito e a aplicação prática dos grafos pode abrir portas para você em *startups* inovadoras ou em grandes corporações que buscam otimizar e inovar seus processos e serviços. A capacidade de modelar, analisar e resolver problemas usando grafos é uma habilidade altamente valorizada no mercado. Empresas líderes mundiais em tecnologia, como Google, Facebook, Amazon e muitas outras, regularmente enfrentam desafios complexos que requerem um entendimento dos grafos.

Por isso, convido você a mergulhar ainda mais nesse mundo. Os exercícios propostos durante a disciplina são uma excelente forma de solidificar o conhecimento adquirido e preparar-se para os desafios práticos que você certamente encontrará em sua carreira profissional.

Até logo!

| Referências

TENENBAUM, A. M. **Estrutura de dados usando C**. São Paulo: Makron Books, 1995.

